

CS 505 Fall 2026 — Lab 1

32-bit Adder Design Space Exploration

Reproducible implementation and experimental report

October 04, 2026

1 Designs, experiment method, and verification

The experiment compares the supplied ripple-carry adder with two new standalone implementations, Kogge–Stone and Brent–Kung. Each has exactly the supplied `adder32(a,b,cin,sum,cout)` interface. Every design is evaluated both as a combinational adder and inside the supplied 32-bit synchronous counter, using the supplied ASAP7 LVT TT, Nangate45 typical, and Sky130HD TT cell libraries. This gives 18 configurations. The optional NAND-only Nangate library is excluded.

Both new designs first form $p_i = a_i \oplus b_i$ and $g_i = a_i b_i$. Prefix groups combine as $(G_H, P_H) \circ (G_L, P_L) = (G_H \vee (P_H \wedge G_L), P_H \wedge P_L)$. Kogge–Stone uses five prefix stages with distances 1, 2, 4, 8, and 16 (129 prefix-combine nodes). Brent–Kung uses five reduction stages followed by four distribution stages at distances 8, 4, 2, and 1 (57 combine nodes). Complete prefixes provide $c_0 = cin$, $c_{i+1} = G_{i:0} \vee (P_{i:0} \wedge cin)$, $sum_i = p_i \oplus c_i$, and $cout = c_{32}$. The original bit propagate values are retained for the sum. The RTL uses explicit combinational stages rather than an inferred generic addition operator. These node counts describe the source prefix graphs, not the number of cells after optimization and mapping.

Yosys maps each design into the selected library. OpenSTA first determines the minimum timing period, then estimates power with input activity 0.2 transitions per minimum period. Verilator simulates the mapped netlist using the supplied cell simulation models and produces the VCD used in a separate power analysis. Thus correctness, area, timing, and power refer to the same mapped implementation. All 18 mapped simulations passed; the analyzer also requires nonzero VCD activity annotation and complete reports.

The supplied testbenches were modified for this experiment. Their source files are included in `submission.zip`; Appendix C summarizes their workloads and verification. All 18 mapped PASS results and the reported VCD-based power estimates use these enhanced benches, with identical stimuli across architectures and technologies. Using the instructor’s original stimulus can produce different VCD power estimates.

The enhanced adder bench compares all 33 result bits, including carry-out, uses directed full/partial carry chains and overflow cases with both values of carry-in, and then 100,000 deterministic pseudorandom vectors. The counter bench checks reset results after clock edges, normal counting, and reset during counting. Icarus runs both benches against every RTL architecture. In addition, six Yosys SAT proofs pass: complete 33-bit addition for all 2^{65} combinations of inputs in each architecture, and the counter transition/reset property from an unconstrained register state for each architecture. The latter covers wraparound from `0xffffffff` without simulating billions of cycles.

The adder stimulus interval is 0.9855 ns (quantized to the testbench 1 ps precision); the counter clock period is 10 ns. These VCD workloads are fixed across architectures and technologies. Their activity rates differ from the probabilistic model, which uses each circuit’s minimum period. Consequently, differences between the two power columns reflect both activity assumptions and rate assumptions, not only circuit implementation. Specify timing checks are ignored by Verilator; this is functional gate-level activity, not a post-layout, delay-annotated measurement of silicon glitches.

1.1 Units and derived metrics

Timing reports use ps for ASAP7 and ns for Nangate45/Sky130HD; the analyzer reads `sta/units.log` and converts to seconds before calculations. OpenSTA’s `report_power` table is already in watts, even when `report_units` shows a different Liberty power unit. Area is the sum of mapped cell areas in the library, reported as μm^2 ; it does not

include placement utilization or routed wire area. Let T be the minimum period in seconds, A mapped area, and P total power. Then throughput is $f = 1/T$, performance/area is f/A , and energy/operation is $E = PT$. Conventional throughput/watt is $f/P = 1/(TP)$ in operations/joule. The handout instead explicitly specifies T/P for “performance per Watt”; both quantities are retained with distinct names in the results and tables. Neither is silently substituted for the other.

The provided screenshot removes the official-template CSV instruction. The generated `results.csv` is therefore an auxiliary reproducibility artifact with our own explicit headers, not a purported official template.

2 Measured PPA results

The tables report total power (internal, switching, and leakage combined). All rows passed their mapped testbench. Energy here follows the handout’s PT definition using the minimum period. VCD power uses activity recorded at the fixed simulation workload rate; both power runs retain the minimum-period STA clock constraints. Its PT value is an assignment-defined metric, not the directly measured energy per simulated transaction.

2.1 Standalone adders

ASAP7 (7 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	11.36	0.8077	27.87	27.37	22.51	22.11
Kogge–Stone	19.39	0.3221	120.8	46.77	38.89	15.06
Brent–Kung	15.82	0.416	76.77	36.71	31.93	15.27

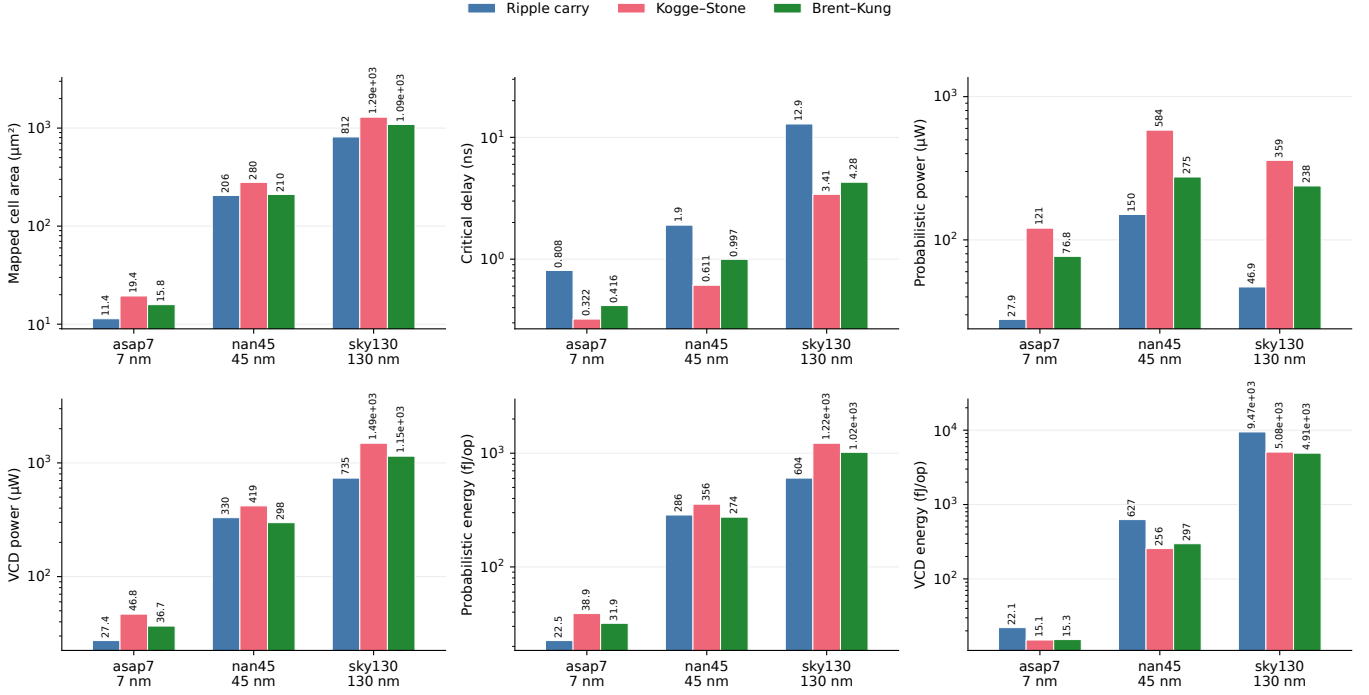
Nangate45 (45 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	205.6	1.902	150.4	329.8	286.1	627.3
Kogge–Stone	279.6	0.6107	583.6	418.7	356.4	255.7
Brent–Kung	210.4	0.9972	274.8	298	274	297.2

Sky130HD (130 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	812	12.88	46.92	735.2	604.3	9469
Kogge–Stone	1292	3.407	359.3	1490	1224	5075
Brent–Kung	1089	4.282	237.9	1146	1019	4909

32-bit standalone adders: mapped-cell comparisons



2.2 Counters

ASAP7 (7 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	18.56	0.3911	110.3	97.61	43.15	38.17
Kogge–Stone	18.62	0.3471	124.1	109.8	43.09	38.13
Brent–Kung	18.34	0.3679	117.1	103.7	43.08	38.15

Nangate45 (45 nm)

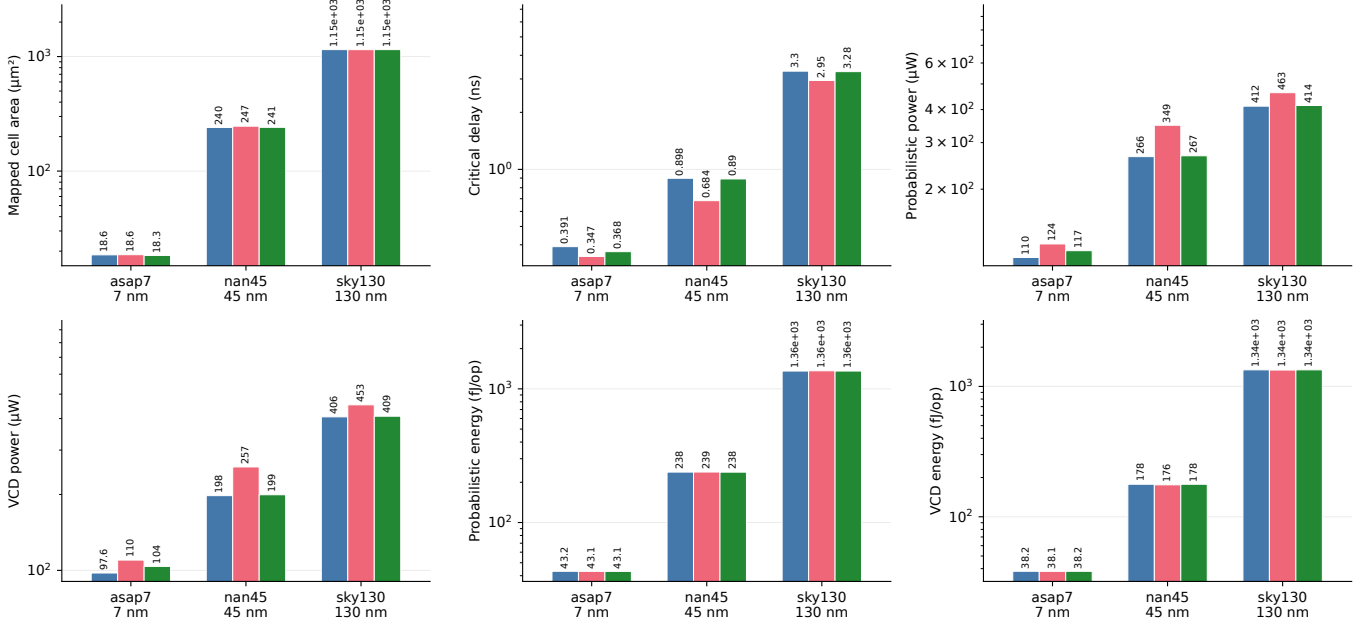
Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	240.5	0.8977	265.5	197.9	238.3	177.6
Kogge–Stone	246.6	0.6837	348.9	257.2	238.5	175.8
Brent–Kung	241	0.8902	267.5	199.5	238.1	177.6

Sky130HD (130 nm)

Architecture	A (μm^2)	T (ns)	P_{prob} (μW)	P_{VCD} (μW)	E_{prob} (fJ)	E_{VCD} (fJ)
Ripple-carry	1150	3.298	412	406.4	1359	1340
Kogge–Stone	1149	2.946	463.3	453.5	1365	1336
Brent–Kung	1150	3.279	414.5	408.7	1359	1340

32-bit counters: mapped-cell comparisons

■ Ripple carry ■ Kogge-Stone ■ Brent-Kung



Logarithmic y axes. Energy = critical delay × power; excludes physical implementation and routing.

3 Qualitative questions

3.1 What is a netlist, and why analyze it instead of the Verilog source?

The original Verilog describes operations and logic connections, but does not select technology-specific cells with known area, delay, and power. Yosys optimizes this logic and maps it to library cells; `synth/netlist.v` lists those cells and their connections. PPA analysis combines this mapped circuit with the library's physical models, so it reflects the selected technology and synthesis optimizations. The source alone does not provide that information.

3.2 Two reasons an implementation might run faster than STA suggests

- STA may consider a path that valid inputs can never activate. If this is proved for the reported longest path, it can be excluded; the longest remaining valid path then sets the timing limit and may allow a faster clock.
- Actual gates may run faster under more favorable voltage, temperature, or manufacturing conditions than the library model assumes.

These possibilities were not demonstrated in this experiment.

3.3 How can the original testbenches be improved?

Both benches were improved:

- **Adder:** The original connects `cout` but checks only `sum`; random tests can miss long carry chains. The enhanced bench checks `{cout, sum}`, every carry-chain length, overflow, and both carry-in values, with reproducible pseudorandom inputs.
- **Counter:** The original checks counting after startup reset. The enhanced bench also checks that the output is zero during reset, reapplies reset after 50,000 increments, and verifies counting restarts from zero. A reference count checks the output every cycle.

Both benches use `$fatal` to return a nonzero status on failure. Separate formal proofs cover all adder inputs and counter states, including wraparound; mapped simulation checks library integration.

3.4 Where does VCD activity come from, and what affects its power?

VCD (Value Change Dump) records signal values and the times at which they change. Verilator creates this file while simulating the mapped netlist with the testbench; OpenSTA reads the circuit’s recorded activity to estimate power. The commands and settings mean:

- `$dumpfile(vcd_name)` selects the output filename; `vcd_name` holds that filename.
- `$dumpvars(0)` starts recording signals. The zero requests all levels of nested modules, including the circuit being tested.
- `next_random` is a function that generates repeatable 32-bit pseudorandom numbers for the adder inputs.
- `apply_vector` is a task that sets the adder’s two operands and carry-in, waits, and checks the outputs.
- `SAMPLE_INTERVAL_NS` sets the delay between adder tests: 0.9855 ns, rounded to 0.986 ns by the simulator’s time precision.
- `always #5` toggles the counter clock every 5 ns, giving a 10 ns clock period.

Input patterns determine which signals switch. Shorter input intervals or clock periods usually increase dynamic power, the power associated with switching. Reset frequency and simulation length change the recorded workload; longer counter runs can capture rare changes in higher bits. The dump commands record activity rather than drive circuit inputs.

3.5 How did AI help with this assignment? (Optional)

AI helped me configure the experiment environment faster by explaining tool paths, dependencies, and build commands. It also helped me understand Verilog and digital circuit principles, especially carry propagation and the Kogge–Stone and Brent–Kung prefix networks. Step-by-step explanations connected RTL behavior to area and delay tradeoffs.

4 Quantitative questions

4.1 Three slowest unique start/end pairs

[Appendix A](#) lists the three slowest distinct start/end pairs for all 18 configurations, ranked by the worst slack for each pair. A unique pair requires a distinct combination of startpoint and endpoint; sharing only the endpoint does not make two pairs the same.

All 18 `sta/timing-longest-paths.rpt` files repeat pairs. Different gate/pin routes, rise/fall transitions, and conditional gate-delay models (delay cases depending on other inputs) can share the same endpoints. These conditions are not printed, so some entries can even have identical printed details.

For example, the ASAP7 ripple-carry report’s 1,000 entries all have `b[0] → sum[31]`. Querying each source separately in `timing-unique-paths.rpt` supplies the other pairs. Its three slowest distinct pairs are:

Startpoint	Endpoint	Delay (ns)	Original occurrences
<code>b[0]</code>	<code>sum[31]</code>	0.8077	1000
<code>cin</code>	<code>sum[31]</code>	0.8063	0
<code>a[0]</code>	<code>sum[31]</code>	0.8052	0

Occurrences count entries in the original 1,000-path report; zero means a pair was absent from that capped list. Each pair is counted once when selecting the top three. Counter timing requirements also include flip-flop setup and capture timing.

4.2 Performance versus performance/area and energy

RCA, KS, and BK denote ripple-carry, Kogge–Stone, and Brent–Kung. Raw performance is the estimated maximum frequency $f = 1/T$, where T is the critical-path delay. Performance/area is f/A , where A is mapped cell area; energy/operation is $E = PT$, where P is total power. E_{prob} uses OpenSTA’s statistical switching model, and E_{VCD} uses power estimated from simulated signal activity.

Every library was evaluated with all three adders, giving nine individual results. The following table shows the values used for the correlations:

Technology	Adder	f (GHz)	f/A (MHz/ μm^2)	E_{prob} (fJ)	E_{VCD} (fJ)
ASAP7 (7 nm)	RCA	1.238	109	22.51	22.11
ASAP7 (7 nm)	KS	3.105	160.1	38.89	15.06
ASAP7 (7 nm)	BK	2.404	152	31.93	15.27
Nangate45 (45 nm)	RCA	0.5257	2.557	286.1	627.3
Nangate45 (45 nm)	KS	1.637	5.857	356.4	255.7
Nangate45 (45 nm)	BK	1.003	4.766	274	297.2
Sky130HD (130 nm)	RCA	0.07764	0.09561	604.3	9469
Sky130HD (130 nm)	KS	0.2935	0.2271	1224	5075
Sky130HD (130 nm)	BK	0.2335	0.2145	1019	4909

For each technology, the next table gives the Pearson correlation coefficient r between raw performance and each column’s metric across the three adders within that library. Thus each row summarizes three adder results, rather than comparing the libraries with each other. Values near +1 indicate that the quantities increase together; values near −1 indicate opposite trends; values near zero indicate little linear relationship.

Technology	r : performance/area	r : prob. energy/op	r : VCD energy/op
ASAP7 (7 nm)	0.973	0.998	-0.938
Nangate45 (45 nm)	0.962	0.837	-0.876
Sky130HD (130 nm)	0.983	0.998	-0.954

- **Performance/area:** Strong positive correlation at all three nodes. Faster adders generally provide more performance per unit area.
- **Probabilistic energy:** Nearly perfect positive correlation at 7 and 130 nm, and a less close positive relationship at 45 nm. Faster adders tend to have higher energy/operation under this model.
- **VCD energy:** Strong negative correlation at all three nodes. Faster adders tend to have lower calculated $P_{VCD}T$.

These are overall trends across three designs, not a guarantee for every pair. The energy models use different activity rates: probabilistic power uses each minimum period, while VCD inputs have a fixed rate. Thus $E_{VCD} = P_{VCD}T$ is the assignment’s derived metric, rather than energy measured per VCD test vector.

4.3 Does architecture improvement scale equally at 7 and 130 nm?

Kogge–Stone is the fastest adder and ripple-carry the slowest in these libraries. For each technology, architecture speedup is $S = f_{adder}/f_{RCA} = T_{RCA}/T_{adder}$: it compares architectures within that technology. The frequency columns show absolute performance.

Technology	f_{RCA} (GHz)	f_{KS} (GHz)	KS/RCA speedup	BK/RCA speedup
ASAP7 (7 nm)	1.238	3.105	2.508	1.942
Nangate45 (45 nm)	0.5257	1.637	3.115	1.907
Sky130HD (130 nm)	0.07764	0.2935	3.781	3.008

Kogge–Stone is 2.508 times faster than ripple-carry at 7 nm and 3.781 times faster at 130 nm. The 7 nm speedup factor is 33.67 percent lower. The relative benefits therefore differ between nodes. All three adders have higher absolute

performance at 7 nm in these STA results; a larger relative gain at 130 nm does not make its adders faster than their 7 nm counterparts.

RCA propagates carries through a serial chain, while KS uses parallel prefix logic. These structures use different gate types and numbers of loads driven by each gate. Library delay/load models and synthesis cell choices differ, so both architectures need not speed up by the same factor between libraries. These are plausible reasons for the discrepancy; routed-wire effects were not measured.

4.4 Discrepancy between 7 nm counter power estimates

Here “discrepancy” means the absolute gap $D = |P_{prob} - P_{VCD}|$. The table also reports the relative gap $100D/P_{prob}$ as a percentage.

Architecture	P_{prob} (μW)	P_{VCD} (μW)	Absolute gap (μW)	Relative gap (%)
Ripple-carry	110.3	97.61	12.73	11.54
Kogge–Stone	124.1	109.8	14.3	11.52
Brent–Kung	117.1	103.7	13.39	11.43

The absolute gap is smallest for Ripple-carry (12.73 μW) and largest for Kogge–Stone (14.30 μW). The relative gap is smallest for Brent–Kung (11.43%) and largest for Ripple-carry (11.54%).

All three relative gaps are about 11.5 percent, with only 0.10 percentage points between the smallest and largest. The main observation is a consistent difference between estimation methods, with little variation across counter architectures.

Reasons for the discrepancies include:

- **Activity assumptions:** The statistical run assumes input activity of 0.2 transitions per minimum period. The VCD records actual binary counting, where bit changes and carry propagation are correlated.
- **Finite workload and reset:** Each counting segment stops at 50,000, so bits 16–31 remain zero. Reset changes only a few times. These patterns differ from statistical activity assumptions.
- **Activity rates:** The VCD records a 10 ns clock, while both STA runs retain minimum-period clock constraints. The recorded data activity and statistical estimates therefore use different rate assumptions.

These differences affect internal and switching power estimates.

4.5 Counter area devoted to sequential elements

The final statistics in `synth/stdout.log` report total cell area A_{total} and sequential area A_{seq} . Every counter has 32 mapped flip-flops, which store its 32-bit count. The sequential share is $100A_{seq}/A_{total}$ percent.

Technology	A_{seq} (μm^2)	RCA share (%)	KS share (%)	BK share (%)
ASAP7 (7 nm)	11.66	62.84	62.65	63.59
Nangate45 (45 nm)	144.7	60.18	58.68	60.04
Sky130HD (130 nm)	640.6	55.71	55.77	55.71

Across adder designs: The ratio is approximately consistent within each technology; the largest spread is 1.49 percentage points. All three counters use the same 32 flip-flop cells within a library. With $b = 0$ and $cin = 1$, synthesis simplifies each adder into an incrementer (adds one). Small differences in the remaining combinational logic area account for the slight ratio changes.

Across technology nodes: The ratio changes and is higher at 7 nm than at 130 nm in these results. Each library has different relative areas for flip-flops and logic gates, and synthesis maps the logic differently. Keeping 32 flip-flops therefore does not keep their share of total area constant.

5 Rebuild guide

GitHub repository: <https://github.com/Colossus-MKII/cs505-f26-lab1>.

Use the complete project checkout on ilab, with the environment described in the assignment handout.

1. Open the project directory:

```
cd cs505-f26-lab1
```

2. Rebuild all experiments:

```
python3 scripts/run_experiments.py --force
```

This checks RTL correctness, synthesizes and simulates all 18 configurations, and analyzes timing, area, and both power estimates.

3. **submission.zip** contains the two new adder files, the two enhanced testbench files, and **report.pdf**. Data and plots are in **results/**; synthesis, simulation, and STA reports are in **build/**.

6 Appendix

A Three slowest unique endpoint pairs for every configuration

Names are copied from each **sta/timing-unique-paths.rpt**. The table reports the magnitude of worst slack normalized to ns (the timing requirement) and the number of occurrences of that endpoint pair in the original capped **timing-longest-paths.rpt**. Zero means that pair did not appear within the first 1,000 raw paths, although it is present in the complete unique-pair report.

A.1 adder32: Ripple-carry, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[31]	0.8077	1000
cin	sum[31]	0.8063	0
a[0]	sum[31]	0.8052	0

A.2 adder32: Kogge–Stone, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
a[0]	sum[30]	0.3221	48
b[0]	sum[30]	0.3207	48
cin	sum[30]	0.3196	48

A.3 adder32: Brent–Kung, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
a[0]	sum[31]	0.416	193
b[0]	sum[31]	0.4147	155
cin	sum[31]	0.4137	124

A.4 adder32: Ripple-carry, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[31]	1.902	955
a[0]	sum[31]	1.895	43
cin	sum[31]	1.891	2

A.5 adder32: Kogge–Stone, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[30]	0.6107	183
a[0]	sum[30]	0.6068	171
a[0]	sum[31]	0.5991	33

A.6 adder32: Brent–Kung, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[31]	0.9972	209
b[0]	sum[30]	0.9959	150
a[0]	sum[31]	0.9934	207

A.7 adder32: Ripple-carry, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
cin	cout	12.88	2
a[0]	cout	12.87	2
b[0]	cout	12.86	2

A.8 adder32: Kogge–Stone, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	sum[30]	3.407	4
b[0]	cout	3.406	2
cin	sum[30]	3.403	4

A.9 adder32: Brent–Kung, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
b[0]	cout	4.282	2
cin	cout	4.268	2
a[0]	cout	4.266	2

A.10 counter32: Ripple-carry, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PN0__Q_1	counter_\$_SDFF_PN0__Q	0.3911	4
counter_\$_SDFF_PN0__Q_2	counter_\$_SDFF_PN0__Q	0.3898	4
counter_\$_SDFF_PN0__Q_1	counter_\$_SDFF_PN0__Q_31	0.3883	10

A.11 counter32: Kogge–Stone, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_3	counter_\$_SDFF_PNO__Q_29	0.3471	9
counter_\$_SDFF_PNO__Q_3	counter_\$_SDFF_PNO__Q_31	0.3467	3
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_29	0.3463	9

A.12 counter32: Brent–Kung, ASAP7 (7 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q_31	0.3679	16
counter_\$_SDFF_PNO__Q_3	counter_\$_SDFF_PNO__Q_31	0.3667	16
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_31	0.3649	16

A.13 counter32: Ripple-carry, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	0.8977	4
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	0.8976	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_31	0.8918	8

A.14 counter32: Kogge–Stone, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	0.6837	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_31	0.681	8
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	0.6796	4

A.15 counter32: Brent–Kung, Nangate45 (45 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	0.8902	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	0.8845	4
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q_31	0.8844	8

A.16 counter32: Ripple-carry, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	3.298	4
counter_\$_SDFF_PNO__Q_3	counter_\$_SDFF_PNO__Q	3.273	4
counter_\$_SDFF_PNO__Q_2	counter_\$_SDFF_PNO__Q	3.272	4

A.17 counter32: Kogge–Stone, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q	2.946	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_30	2.944	4
counter_\$_SDFF_PNO__Q_1	counter_\$_SDFF_PNO__Q_29	2.934	4

A.18 counter32: Brent–Kung, Sky130HD (130 nm)

Startpoint	Endpoint	Requirement (ns)	Occurrences
counter\$_\$_SDFF_PNO__Q_3	counter\$_\$_SDFF_PNO__Q	3.279	4
counter\$_\$_SDFF_PNO__Q_1	counter\$_\$_SDFF_PNO__Q	3.278	4
counter\$_\$_SDFF_PNO__Q_2	counter\$_\$_SDFF_PNO__Q	3.277	4

B Both definitions of the requested efficiency metric

Here T/P is the handout’s literal quantity in s/W. Throughput/W is the conventional $1/(TP)$, shown in trillions of operations per joule (TOp/J). The complete CSV also contains throughput, performance/area, and both energies.

B.1 adder32

Technology	Architecture	T/P_{prob}	T/P_{VCD}	f/P_{prob}	f/P_{VCD}
ASAP7 (7 nm)	Ripple-carry	2.898e-05	2.951e-05	44.43	45.24
ASAP7 (7 nm)	Kogge–Stone	2.667e-06	6.887e-06	25.71	66.39
ASAP7 (7 nm)	Brent–Kung	5.419e-06	1.133e-05	31.32	65.49
Nangate45 (45 nm)	Ripple-carry	1.265e-05	5.769e-06	3.495	1.594
Nangate45 (45 nm)	Kogge–Stone	1.047e-06	1.459e-06	2.806	3.911
Nangate45 (45 nm)	Brent–Kung	3.629e-06	3.346e-06	3.649	3.365
Sky130HD (130 nm)	Ripple-carry	0.0002745	1.752e-05	1.655	0.1056
Sky130HD (130 nm)	Kogge–Stone	9.482e-06	2.287e-06	0.817	0.197
Sky130HD (130 nm)	Brent–Kung	1.8e-05	3.735e-06	0.9815	0.2037

B.2 counter32

Technology	Architecture	T/P_{prob}	T/P_{VCD}	f/P_{prob}	f/P_{VCD}
ASAP7 (7 nm)	Ripple-carry	3.544e-06	4.007e-06	23.17	26.2
ASAP7 (7 nm)	Kogge–Stone	2.796e-06	3.16e-06	23.21	26.23
ASAP7 (7 nm)	Brent–Kung	3.143e-06	3.549e-06	23.21	26.21
Nangate45 (45 nm)	Ripple-carry	3.381e-06	4.537e-06	4.196	5.63
Nangate45 (45 nm)	Kogge–Stone	1.959e-06	2.659e-06	4.192	5.688
Nangate45 (45 nm)	Brent–Kung	3.328e-06	4.463e-06	4.2	5.631
Sky130HD (130 nm)	Ripple-carry	8.005e-06	8.117e-06	0.7358	0.7461
Sky130HD (130 nm)	Kogge–Stone	6.359e-06	6.496e-06	0.7327	0.7486
Sky130HD (130 nm)	Brent–Kung	7.911e-06	8.023e-06	0.7358	0.7462

C Testbenches and verification

The two enhanced testbench source files are included in `submission.zip`. Place them at the following paths in the project checkout:

- `src/adder32/adder32-testbench.v`
- `src/counter32/counter32-testbench.v`

Both benches check outputs automatically and terminate with a nonzero status on failure. The rebuild command in Section 5 runs both against every RTL architecture and every mapped configuration.

C.1 Adder testbench

`adder32-testbench.v` checks all 33 result bits, including carry-out. It uses 106 directed cases and 100,000 deterministic pseudorandom cases with seed `0x505f2601`. The directed cases cover carry chains, overflow, and both carry-in values. Its 0.9855 ns stimulus interval rounds to 986 ps at the declared 1 ps precision.

C.2 Counter testbench

`counter32-testbench.v` checks 100,000 increments against a reference count, plus startup reset and reset during counting. It uses a 10 ns clock and checks four reset cycles. Two counting segments of 50,000 increments verify that counting restarts correctly after reset.

These workloads generated the VCD activity used for the reported power estimates. Verilator simulates the mapped netlists with these same benches; `+vcd=filename.vcd` enables waveform dumping for OpenSTA.

C.3 Compatibility with the original instructor benches

The original bench files were recovered without edits from course repository revision `92a4172`. Verilator ran each original bench against the current standalone RTL of both submitted adders. All four checks passed. This is separate from the 18 technology-mapped simulations using the enhanced benches.

Architecture	Original testbench	Result
Brent-Kung	adder32	PASS
Brent-Kung	counter32	PASS
Kogge-Stone	adder32	PASS
Kogge-Stone	counter32	PASS

Exact commands, source hashes, and logs are recorded in `build/original-testbenches/verification.json`.